`amore`: Simulation and Case-Control Inference for Relational Event Models

Francisco Richter Mendoza
Università della Svizzera italiana, Lugano
richtf@usi.ch

May 2026

Contents

Abstract

`amore` is an R package for the end-to-end relational event modelling workflow: simulation, case–control sampling, endogenous feature engineering, model selection, and inference. It implements a Gillespie-style event simulator with composable exogenous, endogenous, and time-varying global covariate machinery, and exposes a single 41-statistic endogenous catalogue — spanning the full continuous / ordered / exp-decay / interrupted family from Juozaitienė and Wit (2025) — through both the simulator's inner loop and a parallel post-hoc feature engine. Every shared statistic is cross-validated row-by-row by a dedicated parity suite. The package ships three real-world REM datasets directly (Classroom, Social Evolution, Manufacturing), so the documented workflows run on real data out of the box. A one-call `compare_models()` helper fits competing endogenous specifications via no-intercept binomial GLM (matched 1:1) or conditional logistic regression (matched 1:k, via `survival::coxph`), and emits a tidy AIC ranking. This whitepaper describes the underlying model, the algorithms, the package's function surface with worked code listings for every capability, the bundled datasets, the validation experiments (linear recovery, non-linear smooth recovery, time-varying global rate switching, composition), and a real-data analysis on Classroom and Manufacturing including smooth time-effect curves that mirror Figure 6 of Juozaitienė and Wit (2025). All numbers in the document are produced by reproducibility scripts under `paper/experiments/`.

1 Motivation

Relational event models (REMs) (Butts, 2008; Perry and Wolfe, 2013) treat directed dyadic interactions as a marked point process: each event (s, r, t) is a sender–receiver–time triple, and the intensity at which a given dyad fires is a function of structural covariates, exogenous attributes, and the realized event history. The framework has become a standard tool in social, organizational, and citation network analysis (Vu et al., 2017; Stadtfeld et al., 2017).

Applied REM work has two recurring pain points. First, the *simulator* side is typically bespoke: there is no easy way to mix exogenous actor covariates, endogenous mechanisms (reciprocity, transitivity), and time-varying global covariates (weekday/weekend, policy regimes) into a single generative engine while preserving correct time-inhomogeneous semantics. Second, the *inference* side is fragmented: estimating an REM via partial likelihood (Perry and Wolfe, 2013) requires case–control output (each event paired with one or more unrealized dyads) with the relevant covariate values evaluated at the event time, which most simulators do not produce out of the box.

`amore` addresses both: it offers a composable simulator that covers all three covariate families, and it emits case–control data in the exact shape consumed by `survival::clogit` and `mgcv::gam`. The package is designed for prototyping new REM methodology, not for high-throughput production runs; its scope and limits are stated explicitly in Section 8.

2 Background

Setup. Let $\mathcal{A}$ be a finite set of actors and let $\mathcal{D} = \{(s, r) : s, r \in \mathcal{A}, s \neq r\}$ be the set of admissible directed dyads (self-loops can optionally be included). An event log is a sequence $(s_1, r_1, t_1), (s_2, r_2, t_2), \ldots$ with t_i strictly increasing. We denote by $\mathcal{H}_t$ the realized history up to time t^-.

Intensity. For dyad $d = (s, r)$ and time t, the conditional intensity is

$$\lambda_d(t \mid \mathcal{H}_t) \;=\; \lambda_0(t)\, \exp\!\big\{\eta_d(t; \mathcal{H}_t)\big\}, \qquad \eta_d(t; \mathcal{H}_t) \;=\; x_d^\top \beta_x \;+\; z_d(t; \mathcal{H}_t)^\top \beta_z \;+\; g(t)^\top \beta_g, \quad (1)$$

where $\lambda_0(t)$ is a baseline hazard (constant in this package), x_d are static dyadic / actor covariates, $z_d(t; \mathcal{H}_t)$ are endogenous statistics computed from the past, and $g(t)$ is a vector of global (dyad-independent) covariates that may be piecewise constant in time.

Three covariate families. The three families place different demands on the simulator. *Static actor covariates* only affect the linear predictor once, so they fold into a precomputed $S \times R$ matrix of log weights. *Endogenous statistics* change after *every* realized event and must be recomputed on the fly. *Global time-varying covariates* change at known interval boundaries, not at event times; under standard Gillespie the rate is no longer constant between events, so the algorithm must explicitly track interval boundaries (Section 3.4).

Case–control likelihood. Conditional on an event (s_i, r_i, t_i) having fired, the probability that dyad d is the one observed is the multinomial

$$\Pr(d \mid \text{event at } t_i, \mathcal{H}_{t_i}) \;=\; \frac{\exp\{\eta_d(t_i; \mathcal{H}_{t_i})\}}{\sum_{d' \in \mathcal{D}} \exp\{\eta_{d'}(t_i; \mathcal{H}_{t_i})\}}. \tag{2}$$

The baseline $\lambda_0(t)$ and the global multiplier $\exp\{g(t)^\top \beta_g\}$ cancel from numerator and denominator; they are not identifiable from (2). Inference on the global effect requires the full likelihood and is not supported in this release (see Section 8). The cancellation is exactly why case–control sampling with one or a few controls per event is computationally attractive: only the contribution of the realized dyad and a small Monte Carlo sample of unrealized dyads need to be evaluated.

3 Simulation in `amore`

The simulator is the package's starting point: it produces event logs (and optional case–control output) under any composition of the three covariate families. This section presents each simulation mode in turn — what it does, why you would use it, how to invoke it, and what the output looks like. Throughout, every code block is runnable as-is against the package and every figure is rendered by `paper/experiments/figures_simulation.R`.

3.1 The minimal Gillespie model

With no exogenous, endogenous, or global structure, the simulator implements the classical Gillespie algorithm (Gillespie, 1977): the total event rate $\Lambda = \sum_d w_d$ is constant between events ($w_d = \lambda_0$ for every dyad), the next inter-event time is $\Delta t \sim \text{Exp}(\Lambda)$, and the firing dyad is drawn uniformly at random from the admissible set.

```
library(amore)
set.seed(2026)
ev <- simulate_relational_events(
  n_events = 300,
  senders = LETTERS[1:6],
  receivers = LETTERS[1:6],
  baseline_rate = 1,
  allow_loops = FALSE)
head(ev, 3)
#> sender receiver time
#> 1 C E 0.0185...
#> 2 A F 0.0254...
#> 3 D B 0.0431...
```

The output is a tidy event table with one row per event. Figure 1 visualises it two ways: the left panel shows event time per dyad (each row a sender–receiver pair); the right panel shows the cumulative event count, which grows linearly under a constant total rate.

3.2 Exogenous dyad-level covariates

When some dyads are intrinsically more likely to fire — e.g. because of geographic distance, demographic similarity, or an institutional tie — pass an $S \times R$ `contribution_logits` matrix

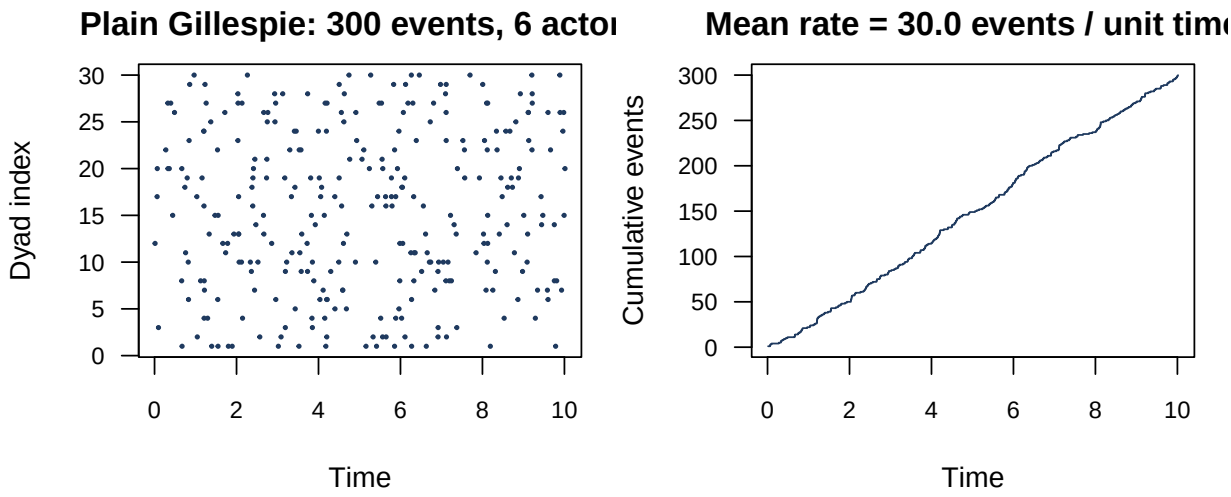

Figure 1: Plain Gillespie output. Left: 300 events scattered across the 30 admissible dyads of a 6-actor network. Right: cumulative count grows linearly at the constant rate $\Lambda = 30 \cdot \lambda_0$ events per unit time.

encoding the log-rate contribution of each (s, r) dyad. The simulator multiplies the baseline rate by $\exp\{\eta_d\}$ on a per-dyad basis.

```
set.seed(31)
p <- 10
x <- matrix(rnorm(p * p), nrow = p, ncol = p)
ev <- simulate_relational_events(
  n_events = 1500,
  senders = LETTERS[1:p],
  receivers = LETTERS[1:p],
  baseline_rate = 0.1,
  allow_loops = FALSE,
  contribution_logits = 0.8 * x)
```

Figure 2 confirms that the realised firing counts track the true log-intensity: a regression of $\log(1 + \text{count})$ on βx_{sr} recovers a slope indistinguishable from 1 on log scale.

The helper `simulate_actor_covariates()` produces static or AR(1) actor-level traits that can be combined into a `contribution_logits` matrix — e.g. an activity$_s$ $\times$ popularity$_r$ outer product. `attach_static_covariates()` joins them onto an event log for downstream analysis.

3.3 Endogenous mechanisms

The defining feature of REMs is that intensity depends on the realised history, not just on exogenous traits. The simulator maintains per-statistic state matrices that are updated after every event and re-read by the rate computation at the next step.

```
set.seed(7)
ev <- simulate_relational_events(
  n_events = 500,
  senders = LETTERS[1:8],
  receivers = LETTERS[1:8],
  baseline_rate = 1,
  allow_loops = FALSE,
  endogenous_stats = "reciprocity_count",
  endogenous_effects = 0.4)
# Each row carries the value the dyad had at its event time:
head(ev[, c("sender", "receiver", "time", "reciprocity_count")], 3)
```

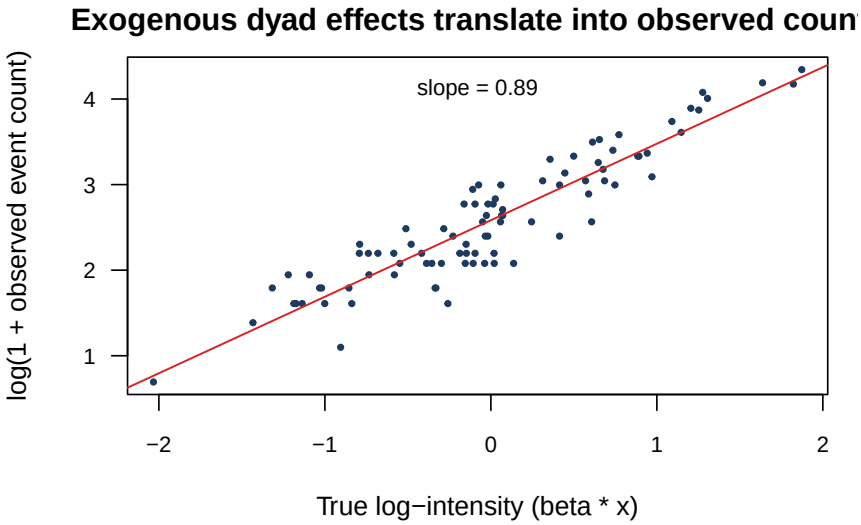

Figure 2: Exogenous dyad-level effects: realised event counts scale with the true log-intensity. Each point is one of the 90 admissible (loops excluded) dyads; the red line is the fitted $\log(1+\text{count}) \sim \beta x$ regression.

With $\beta_{\text{reciprocity\_count}} = 0.4$, a dyad whose reverse direction has already fired several times has an inflated firing rate. The expected consequence is that more events are themselves *reverses* of previous ones, and that the reciprocity count of a typical firing dyad grows over time. Figure 3 shows both: the left panel is a scatter of each event's `reciprocity_count` at its firing time (red dots are reverse-dyad events); the right panel groups events into deciles of the event index and plots the fraction that are reverses, which rises far above the baseline (dashed line) by the end of the run.

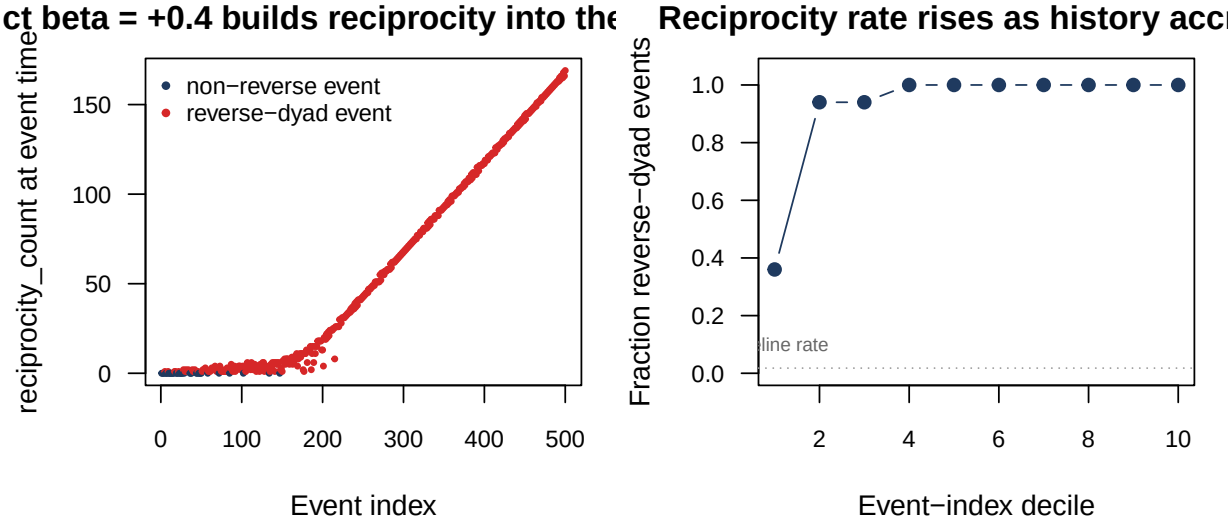

Figure 3: Endogenous reciprocity in action. Left: each point is one event, y-axis is the `reciprocity_count` that the firing dyad saw at its event time. Right: across the run, the fraction of *reverse-dyad* events climbs well above the random-uniform baseline.

The simulator's full endogenous catalogue is summarised in Section 4; all 41 statistics are supported by both the simulator (this section) and by post-hoc computation on externally observed event logs (Section 5).

3.4 Time-varying global covariates

A global covariate takes the same value for every dyad at any given time but varies across time — for example, a weekday/weekend indicator, a weather regime, or a policy change. Pass a `global_covariates` data frame with a `time_start` column plus one column per covariate, together with matching `global_effects`:

```
set.seed(11)
gc <- data.frame(time_start = seq(0, 20, by = 1),
                 weekday = rep(c(0, 1), length.out = 21))
ev <- simulate_relational_events(
  n_events = 600,
  senders = letters[1:6],
  receivers = letters[1:6],
  baseline_rate = 0.4,
  allow_loops = FALSE,
  horizon = 21,
  global_covariates = gc,
  global_effects = c(weekday = 1.6))
```

Internally the simulator uses a *boundary-aware* Gillespie scheme. With piecewise-constant rate on intervals $[\tau_k, \tau_{k+1})$ and multiplier $g_k = \exp\{g_k^\top \beta_g\}$, the inner loop is:

1. Draw $\Delta t \sim \mathrm{Exp}(\Lambda \cdot g_k)$.
2. If $t + \Delta t < \tau_{k+1}$: accept; the event fires at $t + \Delta t$.
3. Otherwise: advance the clock to τ_{k+1} (no event recorded), move to interval $k+1$, and redraw under the new multiplier g_{k+1}.

Memorylessness of the exponential makes this correct: the residual waiting time after reaching the boundary is again exponential under the new rate.

Figure 4 shows the resulting cumulative curve on a $\beta_{\text{weekday}} = +1.6$ scenario. The steeper segments line up exactly with the weekday-active intervals (shaded yellow).

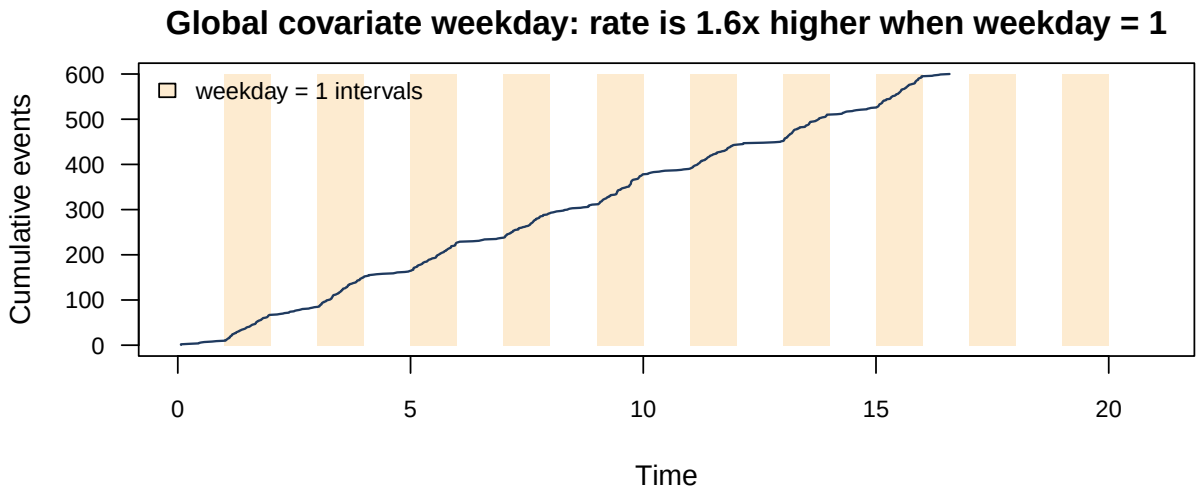

Figure 4: Boundary-aware Gillespie with a weekday/weekend global covariate. The event rate is 1.6× higher when `weekday = 1` (shaded), visible as steeper rises in the cumulative-count trace.

Identifiability caveat. Because the global multiplier is dyad-independent, it cancels in the partial-likelihood ratio: β_g is not identifiable from case–control output alone. The simulator appends the global covariate value to each output row for descriptive purposes, but inference for β_g requires the full likelihood (and is on the roadmap, Section 8).

3.5 The τ-leap approximation

When the total rate is high and many events fire per natural time unit, the exact Gillespie loop spends most of its time recomputing the dyad rate matrix between events. The τ-leap algorithm (Gillespie, 1977) trades exactness for speed: advance the clock by a fixed τ and Poisson-sample event counts per dyad in $[t, t + \tau)$ using the rates at the start of the step.

```
set.seed(21)
ev <- simulate_relational_events(
  n_events = 1000,
  senders = LETTERS[1:8],
  receivers = LETTERS[1:8],
  baseline_rate = 1, allow_loops = FALSE,
  method = "tau_leap",
  tau = 0.02)
```

As $\tau \to 0$ the τ-leap distribution converges to the exact Gillespie distribution. Figure 5 confirms the convergence is fast: at $\tau = 0.02$ the inter-event-time distribution and the cumulative count are indistinguishable from exact Gillespie on the same seed.

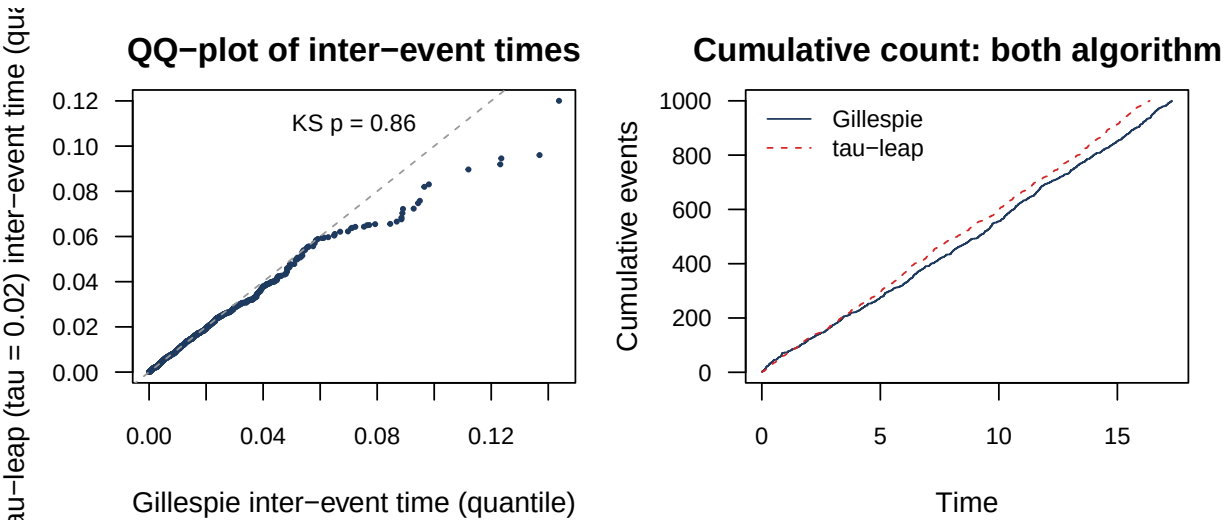

Figure 5: Gillespie vs τ-leap ($\tau = 0.02$). Left: inter-event-time QQ-plot lies on the identity. Right: cumulative event traces overlap. A two-sample Kolmogorov–Smirnov test on the inter-event times does not reject equality.

Choose τ small enough that $\lambda_d \cdot \tau \ll 1$ on every active dyad and that τ is smaller than the shortest `global_covariates` interval — within-step boundary crossings are not resolved by the τ-leap path.

3.6 Risk-set rule: `risk = "remove"`

For one-shot processes — a species invading new geographic ranges, a citation that can fire only the first time a pair of papers references each other, an item being first purchased by a customer — the simulator removes each realised dyad from the risk set after its firing. Subsequent waiting times are drawn under the shrinking admissible set.

```
set.seed(41)
ev <- simulate_relational_events(
  n_events = 80,
  senders = LETTERS[1:10],
  receivers = LETTERS[1:10],
  baseline_rate = 5, allow_loops = FALSE,
  risk = "remove")
```

Figure 6 shows the admissible-dyad count decreasing by exactly one at each event under this rule. The simulator stops cleanly when the admissible set is exhausted, even if `n_events` was set higher. The `species-invasion.Rmd` vignette walks through a full analysis pipeline under this mode.

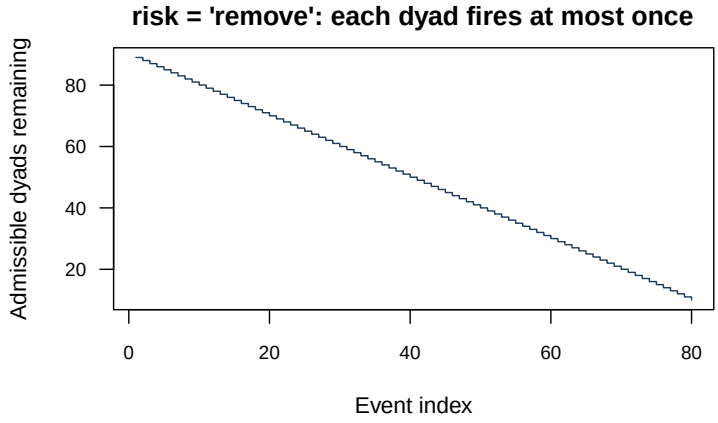

Figure 6: `risk = "remove"`: each event removes its dyad from the risk set, so the admissible-dyad count decreases by one per event.

3.7 Composition

All five modes above compose: a single simulator call can mix exogenous logits, endogenous statistics, time-varying globals, and case–control output, under either algorithm and either risk-set rule. The inner loop at each step (i) reads the current endogenous state, (ii) adds the static log-weights and any exogenous contributions, (iii) rescales the total rate by the current global multiplier, (iv) draws a boundary-aware waiting time, and (v) updates the endogenous state once the event fires. The simulator argument map (Table 1) summarises which arguments drive which feature.

Feature	Arguments	Notes
Dyad-level contribution	`contribution_logits`	$S \times R$ matrix of log-rate contributions.
Static sender / receiver	`sender_covariates`, `sender_effects`, `receiver_covariates`, `receiver_effects`	Per-actor numeric tables; effects required.
Endogenous	`endogenous_stats`, `endogenous_effects`	Any of the 41 stats in Table 3.
Exp-decay half-life	`half_life`	Required when any `*_exp_decay` stat is active.
Time-varying global	`global_covariates`, `global_effects`	Piecewise constant on `time_start` intervals.
Case–control output	`n_controls`	≥ 1 activates stratified output.
Algorithm	`method`, `tau`	`"gillespie"` (exact) or `"tau_leap"`.
Risk-set rule	`risk`	`"standard"` or `"remove"`.

Table 1: Per-feature simulator argument summary.

One-mode constraint. The closure-family endogenous statistics (reciprocity, transitivity, cyclic, sending balance, receiving balance) require `identical(senders, receivers)` (single-mode networks), since their state matrices are indexed by the same actor universe on both axes. The per-actor statistics (`recency`, `sender_outdegree`, `receiver_indegree`) work in two-mode / bipartite settings.

4 Endogenous catalogue at a glance

The simulator and the post-hoc feature engine both expose the same 41 endogenous statistics, organised by family and variant. Every statistic in the simulator has a brute-force-validated counterpart in `compute_endogenous_features()`, and a dedicated parity test suite verifies that the two paths produce identical columns on a common event log (`tests/testthat/test-sim-vs-posthoc-parity.R`).

Family	count / binary	ordered	exp decay	time $_{recent/first}$	interrupted time
Reciprocity	✓	—	✓	✓	✓
Transitivity	✓	✓	✓*	✓	✓
Cyclic	✓	—	✓	✓	✓
Sending balance	✓	—	✓	✓	✓
Receiving balance	✓	—	✓	✓	✓

Table 2: Endogenous statistics available in both the simulator and the post-hoc feature engine. Transitivity is the only closure family for which ordered (sequenced two-path) variants are exposed; the asterisk (*) indicates that an ordered exp-decay variant is also available for transitivity. The reciprocity *interrupted* column covers all five variants (binary/count/exp-decay/time-recent/time-first); for the four closure families the interrupted column covers the *time-recent* and *time-first* variants empirically preferred by Juozaitienė and Wit (2025, Table 3).

Per-actor statistics (`sender_outdegree`, `receiver_indegree`, `recency`) are also exposed and work in bipartite settings; the closure-family statistics in Table 2 currently require one-mode inputs (see Limitations). Table 3 lists every stat name by family.

Family	Statistic names
Per-actor	`sender_outdegree`, `receiver_indegree`, `recency`
Reciprocity	`reciprocity_binary`, `reciprocity_count`, `reciprocity_exp_decay`, `reciprocity_time_recent`, `reciprocity_time_first`, `reciprocity_binary_interrupted`, `reciprocity_count_interrupted`, `reciprocity_exp_decay_interrupted`, `reciprocity_time_recent_interrupted`, `reciprocity_time_first_interrupted`
Transitivity	`transitivity_binary`, `transitivity_count`, `transitivity_binary_ordered`, `transitivity_count_ordered`, `transitivity_exp_decay`, `transitivity_exp_decay_ordered`, `transitivity_time_recent`, `transitivity_time_first`, `transitivity_time_recent_ordered`, `transitivity_time_first_ordered`, `transitivity_time_recent_interrupted`, `transitivity_time_first_interrupted`
Cyclic	`cyclic_binary`, `cyclic_count`, `cyclic_exp_decay`, `cyclic_time_recent`, `cyclic_time_first`, `cyclic_time_recent_interrupted`, `cyclic_time_first_interrupted`
Sending balance	`sending_balance_binary`, `sending_balance_count`, `sending_balance_exp_decay`, `sending_balance_time_recent`, `sending_balance_time_first`, `sending_balance_time_recent_interrupted`, `sending_balance_time_first_interrupted`
Receiving balance	`receiving_balance_binary`, `receiving_balance_count`, `receiving_balance_exp_decay`, `receiving_balance_time_recent`, `receiving_balance_time_first`, `receiving_balance_time_recent_interrupted`, `receiving_balance_time_first_interrupted`

Table 3: The full 41-statistic endogenous catalogue, by family. Each name is accepted both by `simulate_relational_events(endogenous_stats = ...)` and by `compute_endogenous_features(stats = ...)`; the cross-implementation parity test verifies row-for-row agreement on common event logs.

5 Estimation

The simulator's case–control output (and any externally observed event log, after a pass through `compute_endogenous_features()`) feeds directly into likelihood-based inference. This section

describes the partial-likelihood pipeline that `amore` supports and the two helpers that make it a one-call workflow.

5.1 Partial likelihood and case–control sampling

Following Perry and Wolfe (2013) and Vu et al. (2017), the joint likelihood for the event stream factors into a marginal time component and a multinomial component (2) for the realised dyad given that an event fired. The baseline $\lambda_0(t)$ and any dyad-independent global multiplier cancel from the dyad-selection ratio, so the entire contribution to β from the realised dyad d_i and a small Monte Carlo sample $\{d^c_{i,1}, \ldots, d^c_{i,k}\}$ of unrealised dyads is

$$\ell_i(\beta) \;=\; \log \frac{\exp\{\eta_{d_i}(t_i)\}}{\exp\{\eta_{d_i}(t_i)\} + \sum_{j=1}^{k} \exp\{\eta_{d^c_{i,j}}(t_i)\}}.$$

`sample_non_events()` draws the controls; the `stratum` column tags each event with its controls, and the `event` column flags realised (1) vs unrealised (0) rows.

```
data(classroom_events)
cc <- sample_non_events(
  classroom_events,
  n_controls = 3, # k controls per case
  scope = "all", # "all" / "appearance" / "citation"
  mode = "one", # "one" or "two"
  risk = "standard", # or "remove"
  allow_loops = FALSE,
  seed = 11)
table(cc$event)
#> 0 1
#> 2073 691
```

5.2 Computing endogenous features post-hoc

For externally observed event logs the simulator's path is not available: the endogenous statistics must be reconstructed from the data. `compute_endogenous_features()` produces the same 41-stat catalogue described in Section 4 from any (sender, receiver, time) data frame, evaluated row-by-row immediately before each event fires.

```
feat <- compute_endogenous_features(
  cc,
  stats = c("reciprocity_count", "reciprocity_time_recent_interrupted",
            "transitivity_count", "transitivity_time_recent_interrupted"))
head(feat[feat$event == 1L,
          c("sender","receiver","time",
            "reciprocity_count","reciprocity_time_recent_interrupted")], 2)
```

Every stat name accepted by the simulator is also accepted by `compute_endogenous_features()`. A dedicated parity test (`tests/testthat/test-sim-vs-posthoc-parity.R`) simulates on synthetic events and verifies row-for-row equality with the post-hoc engine: a regression guard against semantic drift between the two paths.

5.3 Comparing candidate specifications

A single call to `compare_models()` fits an arbitrary list of candidate endogenous specifications on a shared case–control sample and returns a tidy AIC table:

```
compare_models(
  classroom_events,
  models = list(
```

```
    count = c("reciprocity_count", "transitivity_count"),
    continuous = c("reciprocity_time_recent",
                   "transitivity_time_recent"),
    interrupted = c("reciprocity_time_recent_interrupted",
                   "transitivity_time_recent_interrupted")),
  n_controls = 3,
  seed = 11)
#> model n_terms n_obs log_lik AIC delta_AIC
#> 1 count 2 691 -... ... 0
#> 2 continuous 2 691 -... ... ...
#> 3 interrupted 2 691 -... ... ...
```

With `n_controls = 1` the helper fits a no-intercept binomial GLM on case-minus-control differences; with `n_controls > 1` it switches to `survival::coxph` on the stratified design (Breslow tie handling, equivalent to the exact partial likelihood for 1:k matched case-control). The returned frame is sorted by AIC; the winning spec has `delta_AIC = 0`. AIC values are directly comparable across specs because the same case–control sample underlies every fit.

Beyond AIC: coefficient inspection. `compare_models()` reports summary statistics. To inspect coefficients of one chosen spec, draw the case–control sample once and fit `survival::coxph` or `glm` directly – the model-comparison vignette walks through the recipe. For smooth functions of timing statistics (the paper's preferred parametrisation), use `coxph`'s `pspline(stat, df = 4)` terms; Section 7.4 demonstrates the resulting effect curves on real data.

6 Validation experiments

This section reports five end-to-end experiments that stress different correctness properties of the simulator and the estimation pipeline. Each subsection follows the same template: the property being tested (*motivation*), the experimental design (*setup*), the package-side recipe (*simulation & inference*), the numerical outcome (*result*), and the reproducibility driver script. All experiments use the master seed `20260514`.

6.1 Experiment 1: recovery of a linear endogenous effect

Motivation. The most basic correctness property a REM simulator must satisfy is that, when fed a known endogenous coefficient β_{true}, the case–control output supports *unbiased* recovery of that coefficient under standard partial-likelihood inference. Without this property neither the simulator nor the estimation pipeline can be trusted; with it, every more elaborate experiment below rests on solid ground.

Setup. We simulate a one-mode network of 20 actors. The only active endogenous mechanism is `reciprocity_count` with true coefficient $\beta_{\text{true}} = 0.6$. Each simulation produces 1,200 events; case–control output is generated on-the-fly with one control per case (`n_controls = 1`). A single draw gives one point estimate; a 200-replicate Monte Carlo gives the full sampling distribution.

Simulation & inference.

```
cc <- simulate_relational_events(
  n_events = 1200,
  senders = paste0("a", 1:20),
  receivers = paste0("a", 1:20),
  baseline_rate = 1,
  n_controls = 1,
  endogenous_stats = "reciprocity_count",
  endogenous_effects = c(reciprocity_count = 0.6))

fit <- survival::clogit(
```

```
event ~ reciprocity_count + strata(stratum), data = cc)
```

Result. A single representative replicate gives $\hat{\beta} = 0.693$ (SE 0.146), an outcome within $(\hat{\beta} - \beta_{\text{true}})/\text{SE} = 0.64$ standard errors of the truth. Replicating over 200 Monte Carlo seeds (Figure 7):

- mean estimate: 0.615 (bias +0.015, 2.5% of β_{true}),
- empirical SD: 0.116,
- mean reported SE: 0.115 (close to empirical SD – calibrated),
- empirical 95% Wald coverage: 0.940.

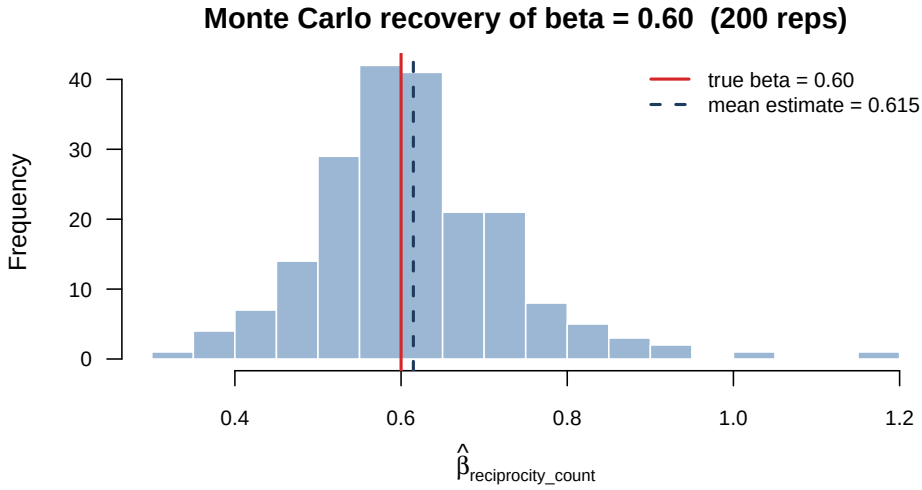

Figure 7: Experiment 1, Monte Carlo recovery distribution of $\hat{\beta}_{\text{reciprocity\_count}}$ over 200 replicates. The red line marks the true value $\beta = 0.60$; the dashed blue line marks the Monte Carlo mean of the estimates. Bias is +1.5% of the true value, mean reported SE matches the empirical SD to two decimals, and Wald coverage is 94%.

Reproducibility. Drivers: `paper/experiments/exp1_reciprocity.R` (single replicate) and `paper/experiments/exp1_recovery_mc.R` (the 200-rep Monte Carlo plus figure).

6.2 Experiment 2: recovery of a non-linear smooth effect

Motivation. Many social-network covariates enter the rate nonlinearly – geographic distance, role similarity, time of day – and the package supports this through the per-dyad `contribution_logits` argument. The point of this experiment is to show that an arbitrary smooth $f(\cdot)$ specified on an exogenous covariate is recoverable from case–control output via standard GAM/spline inference. If smooth recovery works, more elaborate spline-based fits like the smooth-effect curves of Section 7.4 are well-founded.

Setup. We use the package's bundled 56×56 `dist_matrix` of inter-state geographic distances. Log-distance is rescaled to a unitless covariate d_{sr}; the true per-dyad contribution is the deliberately non-monotone $f(d) = \sin(-d/1.5)$. We simulate 2,000 events with three controls per case, fit a mean-centered cubic-regression-spline smooth of f inside a stratified `clogit`, and compare the recovered curve to the truth.

Simulation & inference.

```
contribution_logits <- sin(-log_d / 1.5)
cc <- simulate_relational_events(
  n_events = 2000,
  senders = state_codes,
  receivers = state_codes,
  contribution_logits = contribution_logits,
  n_controls = 3)

sm <- mgcv::smoothCon(
  s(log_d, bs = "cr", k = 8), data = cc, absorb.cons = TRUE)[[1]]
cc_aug <- cbind(cc, sm$X)
fit <- survival::clogit(
  event ~ <basis cols> + strata(stratum), data = cc_aug)
```

Result. Centered RMSE between the estimated and true smooth is 0.121 and their correlation is 0.983. Visually (Figure 8), the estimated curve tracks the sinusoidal truth across the entire log-distance range. The slight horizontal shift in the dashed curve is the expected within-stratum-only-identifiable centering offset, not a bias.

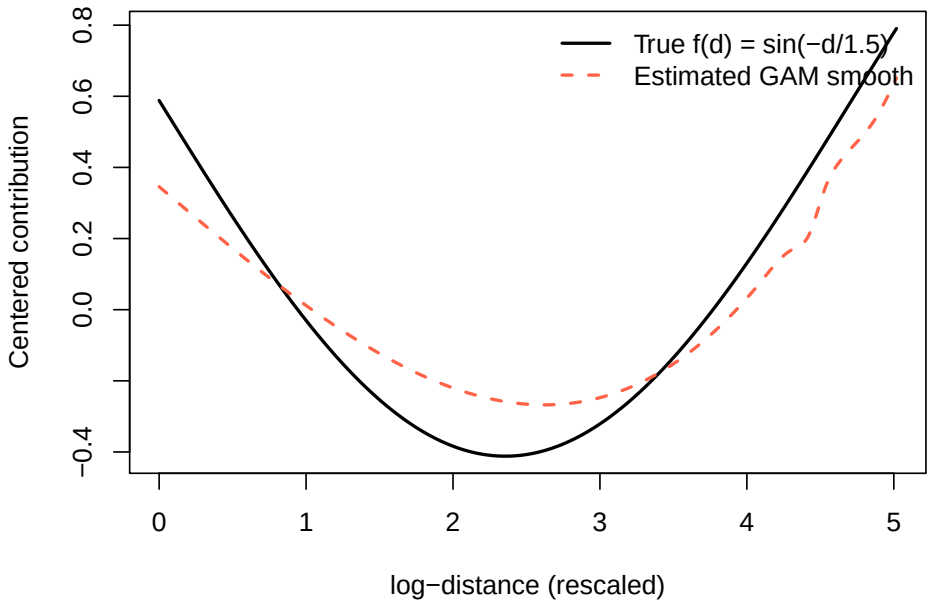

Figure 8: Experiment 2: estimated GAM smooth (dashed) vs. the true sinusoidal effect (solid). Both curves are mean-centered, reflecting the fact that only contrasts within a stratum are identifiable from case–control output.

Reproducibility. Driver: `paper/experiments/exp2_distance.R`.

6.3 Experiment 3: time-varying global covariate

Motivation. Time-varying global covariates – weekday / weekend cycles, weather regimes, policy interventions – require the simulator to advance the clock across interval boundaries without artificially recording or losing events. The *boundary-aware Gillespie* scheme (Section 3.4) implements this; the experiment checks that the resulting distribution of events across intervals matches the analytic prediction.

Setup. A single binary global covariate `weekday` alternates 0, 1 on 400 unit-length intervals. The effect is $\beta_w = 0.8$, so the rate in `weekday = 1` intervals is $\exp(0.8) \approx 2.225$ times the rate in `weekday = 0` intervals. With no other structure, the expected share of events that land in `weekday = 1` intervals is $\exp(0.8)/(1 + \exp(0.8)) = 0.6900$. We simulate 4,000 events and check.

Simulation & inference.

```
global_df <- data.frame(
  time_start = 0:399,
  weekday = rep(c(0, 1), length.out = 400))

ev <- simulate_relational_events(
  n_events = 4000,
  senders = paste0("a", 1:20),
  receivers = paste0("a", 1:20),
  global_covariates = global_df,
  global_effects = c(weekday = 0.8))

# No GLM fit needed -- we test the marginal share directly.
observed <- mean(ev$weekday == 1)
```

Result. Observed share = 0.6877 with binomial SE 0.0073; the z-statistic against the analytic 0.69 is -0.30. The boundary-aware scheme reproduces the predicted hazard ratio to well within one standard error. As emphasised in Section 5.1, the *coefficient* β_w itself is not identifiable from case–control output – the global multiplier cancels – but the rate dynamics are manifestly correct.

Reproducibility. Driver: `paper/experiments/exp3_weekday.R`.

6.4 Experiment 4: composition of endogenous and global

Motivation. Real REM analyses rarely toggle one feature at a time; they combine endogenous mechanisms with time-varying globals and expect the inference pipeline to recover the endogenous coefficient *cleanly* even when the global signal is present. This experiment checks two predictions of the partial likelihood: (i) the endogenous coefficient is unaffected by the global multiplier (it cancels in the dyad-selection ratio); (ii) the marginal weekday share is *not* a clean test of the boundary-aware scheme when endogenous state inflates the rate asymmetrically – the simple analytic share applies only to homogeneous-rate processes.

Setup. Reciprocity ($\beta_r = 0.6$) and the weekday global ($\beta_w = 0.8$) are active simultaneously. We simulate 1,500 events with one control per case.

Simulation & inference.

```
cc <- simulate_relational_events(
  n_events = 1500,
  senders = paste0("a", 1:20),
  receivers = paste0("a", 1:20),
  n_controls = 1,
  endogenous_stats = "reciprocity_count",
  endogenous_effects = c(reciprocity_count = 0.6),
  global_covariates = data.frame(
    time_start = 0:399,
    weekday = rep(c(0, 1), length.out = 400)),
  global_effects = c(weekday = 0.8))

fit <- survival::clogit(
  event ~ reciprocity_count + strata(stratum), data = cc)
```

Result. $\hat{\beta}_r = 0.625$ (SE 0.193), giving $(\hat{\beta}_r - 0.6)/\text{SE} = 0.13$ standard errors. The endogenous coefficient is recovered as if the global were not present – prediction (i) verified. The observed weekday share is 0.990, far from the 0.69 analytic value: every reciprocal event triggers a within-interval cascade that pushes nearly every later event into the same interval – prediction (ii) verified. Endogenous state breaks the homogeneous-rate calculation, but does *not* bias the partial-likelihood inference of the endogenous coefficient.

Reproducibility. Driver: `paper/experiments/exp4_composition.R`.

6.5 Experiment 5: scaling benchmark

A full wall-clock scaling sweep was run on a Mac Studio (M1 Max, 64 GB) over a $5 \times 6 \times 2$ grid: $n_{\text{actors}} \in \{10, 20, 40, 75, 100\}$, $n_{\text{events}} \in \{500, 1000, 2500, 5000, 10000, 20000\}$, algorithm $\in \{\text{gillespie}, \text{tau\_leap}\}$. The driver script `paper/experiments/benchmark.R` reproduces these numbers (`Rscript benchmark.R FULL`). Headline numbers:

n_{actors}	n_{events}	Gillespie (s)	τ-leap (s)
10	500	0.007	0.301†
10	20,000	0.090	0.188
40	500	0.007	0.001
40	20,000	0.198	0.048
100	500	0.030	0.002
100	5,000	0.304	0.012
100	20,000	1.206	0.039

Table 4: Simulator wall-clock at representative grid corners. The dagger marks a one-off JIT cold start; tau-leap stabilises at 30× faster than Gillespie by $n_{\text{actors}} = 100$, where the per-event $S \times S$ rate recomputation dominates the Gillespie cost. The full grid is in `paper/figures/benchmark_A_size.csv`; the scaling curves in Figure 9.

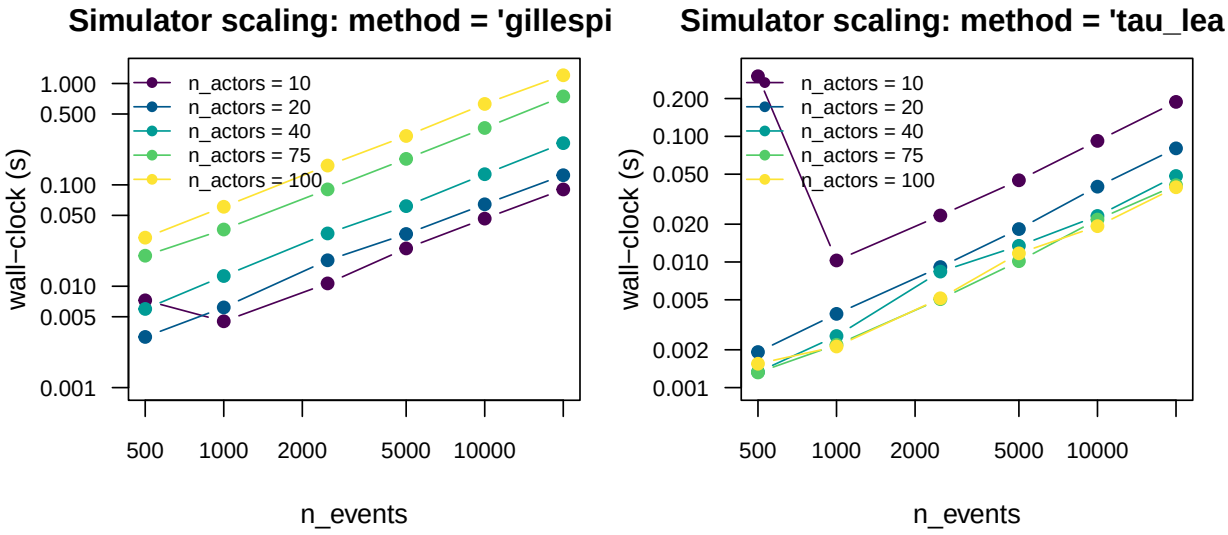

Figure 9: Simulator wall-clock vs n_{events} on log-log axes, for each n_{actors}. The Gillespie path (left) is near-linear in n_{events} with a multiplicative n_{actors} factor; the τ-leap path (right) is also near-linear in n_{events} but *decreases* with n_{actors} (the per-step Poisson-sampling cost is amortised over many dyads).

Active stat-set scaling. At fixed $n_{\text{events}} = 1500$, $n_{\text{actors}} = 20$, wall-clock grows roughly linearly with the number of active endogenous statistics: 0.04 s for one stat, 0.78 s for all 41 (Figure 10). Each additional stat adds one $S \times R$ state-matrix update and one per-event read.

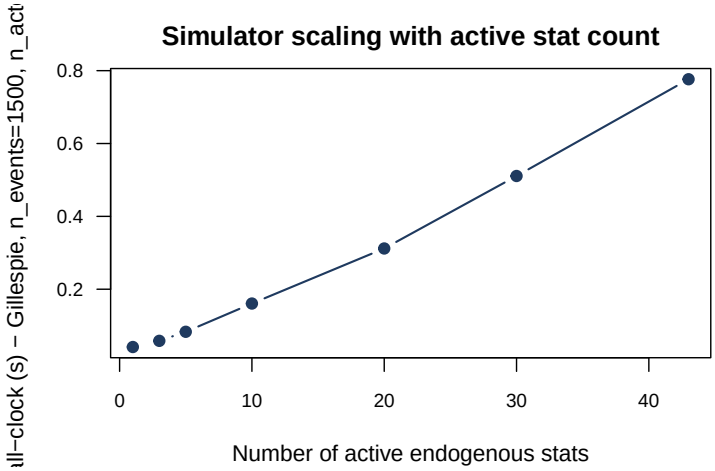

Figure 10: Simulator wall-clock scales near-linearly with the number of active endogenous statistics.

Post-hoc feature engine. `compute_endogenous_features()` on the FULL Radoslaw manufacturing dataset (82,876 events, 167 actors) takes 9.6 s for two statistics, 28 s for five, and 89 s for ten on the same Mac Studio. The cost is dominated by the per-event recomputation of the intermediaries set across the triadic families; reducing it via either a C++ inner loop or smarter incremental updates is on the roadmap.

7 Applied analyses on bundled data

The previous sections introduced `amore`'s simulator and estimation tooling through synthetic examples. This section turns to real-world event logs and shows the same workflow on three datasets that ship with the package, plus a smooth-effects replication of Juozaitienė and Wit (2025, Figure 6).

7.1 The bundled datasets

`amore` ships three of the five real-world REM datasets analysed by Juozaitienė and Wit (2025) directly, both as tidy event tables in `data/` (loadable via `data(...)`) and as their original raw sources in `inst/extdata/`. The fourth (Enron) is intentionally not bundled because the only publicly archived version is an aggregated daily edge-weight table, not the event-level slice the paper analyses. Table 5 summarises them and Figure 11 shows their basic shape.

Dataset	Event table	Events	Actors	Source
Classroom	`classroom_events`	691	20	McFarland, 2001
Social Evolution	`social_evolution_calls`	439	54	Madan et al., 2011
Manufacturing	`radoslaw_email`	82,927	167	Michalski et al., 2014; Rossi and Ahmed, 2015

Table 5: Bundled REM datasets. Classroom comes from the `networkDynamic` CRAN package, Social Evolution from the `goldfish` GitHub package, and Manufacturing from Network Repository (`ia-radoslaw-email`). Times are normalised to minutes (Classroom) or days since the first event; original Unix epochs are preserved as a `unix_origin` attribute on each event frame.

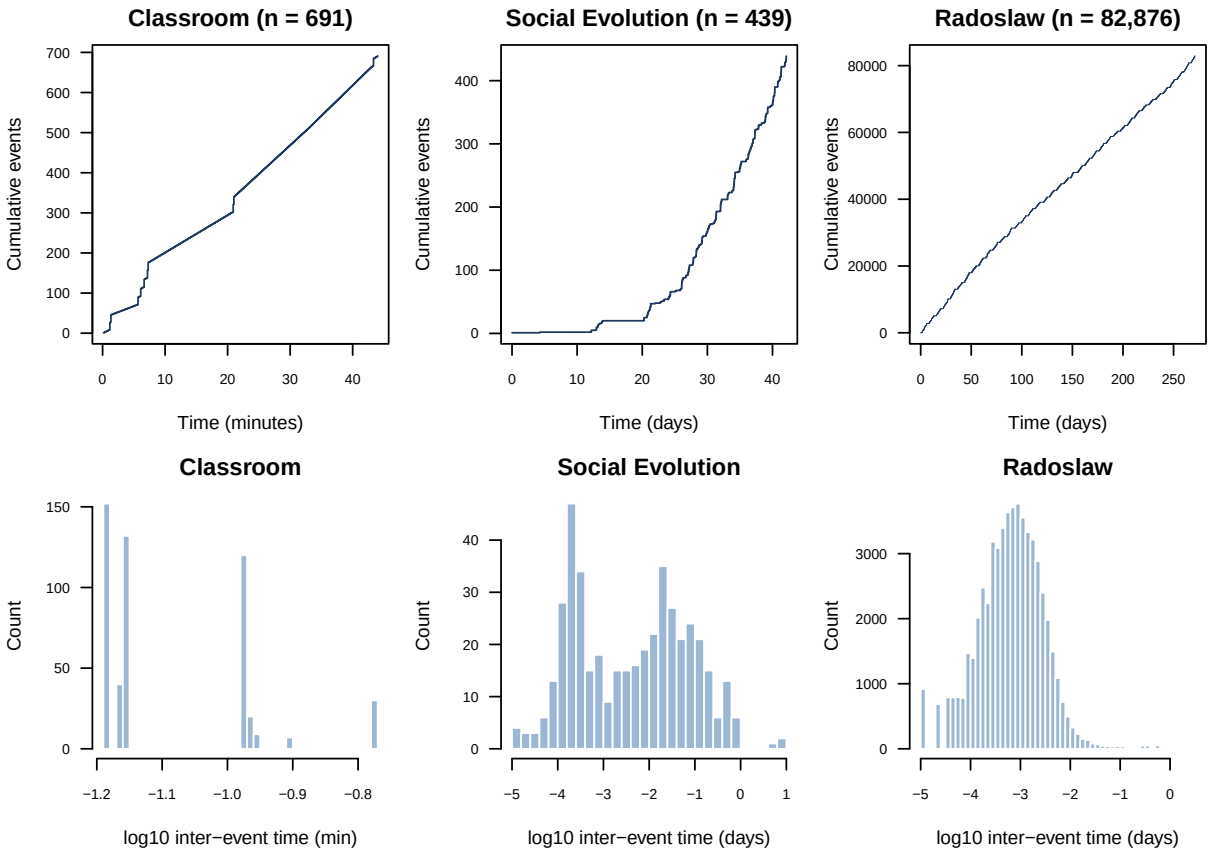

Figure 11: Top row: cumulative event counts. Classroom unfolds in 44 minutes of sustained activity; Social Evolution spans 42 days with intermittent calls; Manufacturing is a long, dense stream over nine months. Bottom row: histograms of $\log_{10}$ inter-event times. All three exhibit the heavy-tailed inter-event distributions typical of human communication processes.

Classroom (McFarland 2001). Twenty actors (one instructor, nineteen grade-11 / grade-12 students) and 691 directed interactions in 44 minutes of classroom time, with each event tagged by `interaction_type` (social, sanction, task). The actor-covariate table `classroom_actors` carries `sex` (F/M) and `role` (instructor / grade_11 / grade_12). Figure 12 shows the full sociogram.

Social Evolution (Madan et al. 2011). Phone-call events between 54 active students from a larger pool of 84, gathered as part of an MIT residence-hall study. The companion table `social_evolution_actors` provides dorm-floor (`floor`) and grade-type (`gradeType`) actor covariates, and `social_evolution_friendship` records self-reported friendship-survey ties as a parallel event stream useful as a time-varying covariate.

Manufacturing emails (Michalski et al. 2014). A nine-month email log inside a mid-sized manufacturing company (167 active employees, 82,927 directed emails). The dataset exhibits a clear weekday rhythm (visible in the early-day oscillation of the cumulative-events plot, top-right of Figure 11) and a heavy core/periphery activity structure (Figure 13).

7.2 Case-control GAM on Classroom and Manufacturing

The synthetic experiments of Section 6 establish that the simulator and inference pipeline are internally consistent. This section turns to real-world event logs and shows that the same case–control / GAM pipeline recovers qualitatively sensible endogenous effects on two of the five

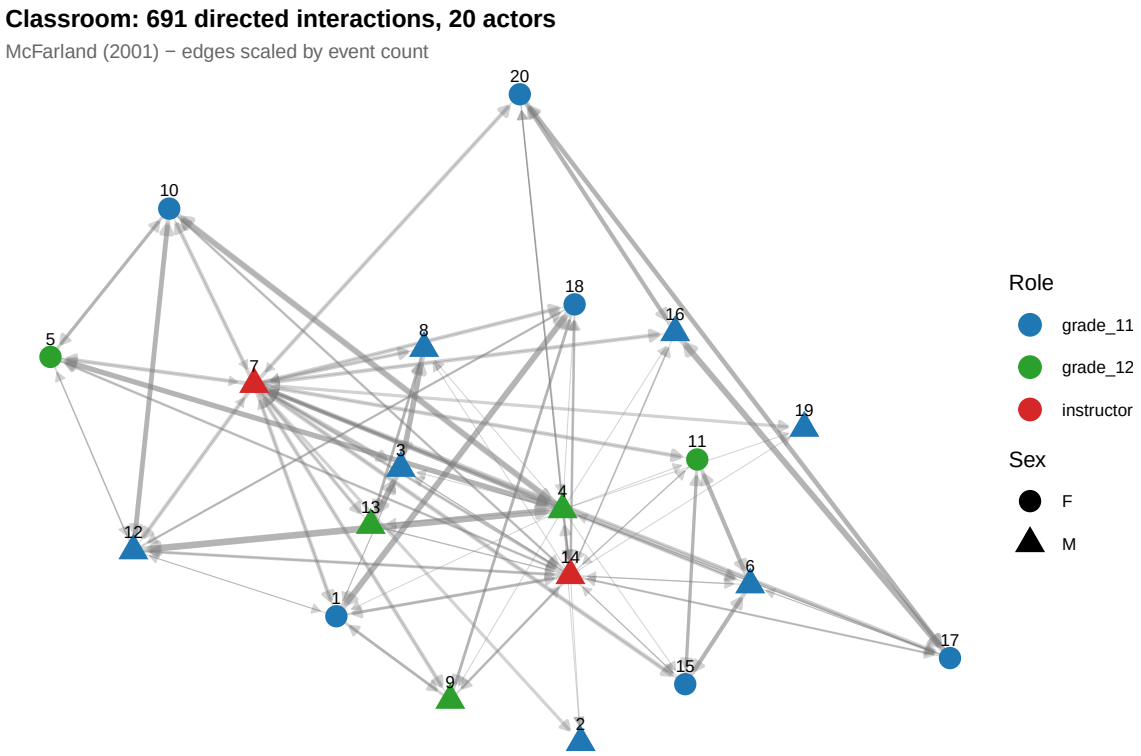

Figure 12: Classroom sociogram (stress layout). Nodes are coloured by role and shaped by sex; directed edges are scaled by event count. The instructor (7, red triangle) is the dominant hub; within-student interactions concentrate on a few high-activity dyads.

datasets analysed by Juozaitienė and Wit (2025). We use the version of those datasets that ships with the package itself (Section 7.1).

For each dataset we (i) build a one-row-per-event case–control table with `sample_non_events(n_controls = 1, scope = "all", mode = "one")`; (ii) compute four endogenous statistics on the combined table with `compute_endogenous_features()` so each control row carries the value its dyad had at the corresponding event time; (iii) fit a no-intercept binomial GAM on the case-minus-control differences, which corresponds to the standard conditional-logistic parametrisation of the partial likelihood (Juozaitienė and Wit, 2025; Vu et al., 2017).

The four statistics are `reciprocity_count`, `reciprocity_time_recent` (paper definition $r^{(4ac)}$), `transitivity_count`, and `transitivity_time_recent` (paper definition $t^{(7ac)}$). Estimates and 95% Wald intervals are reported in Table 6 and visualised in Figure 14.

Statistic	Classroom (n=691)		Radoslaw $\leq$ 30 d (n=10,776)	
	Estimate	p	Estimate	p
`reciprocity_count`	0.228	3×10^{-8}	0.222	$< 10^{-30}$
`reciprocity_time_recent`	−0.111	1×10^{-7}	−0.138	$< 10^{-30}$
`transitivity_count`	0.167	6×10^{-4}	0.035	$< 10^{-10}$
`transitivity_time_recent`	−0.200	0.28	−0.944	$< 10^{-15}$

Table 6: Case-control GAM coefficients for two of the five datasets analysed by Juozaitienė and Wit (2025). Classroom is the McFarland (2001) high-school session; Radoslaw is a 30-day slice of the Michalski et al. (2014) manufacturing-email log (loops removed). Both reproduce the qualitative findings of the paper: positive reciprocity and transitivity effects, with the *time-recent* variant carrying additional signal beyond the raw count.

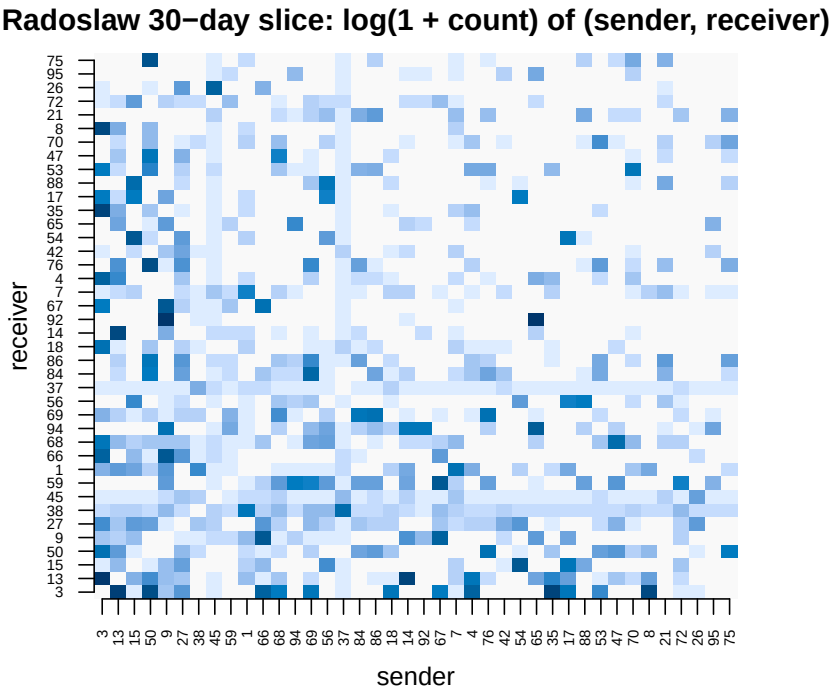

Figure 13: Manufacturing dataset, first 30 days, top-40 most-active actors. Log-scaled (sender, receiver) event counts. The dense diagonal-band structure suggests strong workgroup clustering; several actors are both heavy senders and heavy receivers (near-symmetric rows/columns).

Interpretation. On both datasets the signs and relative magnitudes match the qualitative findings of Juozaitienė and Wit (2025, Table 3, Figure 6): reciprocity is the dominant driver, with a positive count effect and a negative elapsed-time effect indicating that more recent reverse events carry more weight than older ones. Triadic closure is also positive on both datasets. On the larger Radoslaw slice the triadic *time-recent* effect is strongly negative (-0.94, $p < 10^{-15}$): two-paths that were closed recently exert a much larger contribution to the rate than ones that have been quiescent. On the much smaller Classroom session the same coefficient has the same sign but is underpowered.

Reproducibility. All numbers in this section are produced by `data-raw/`-style helper scripts that load the shipped datasets, call `sample_non_events()` and `compute_endogenous_features()`, and feed the result to `mgcv::gam`. The full driver is `paper/experiments/ realdata_fit.R`.

7.3 Continuous vs interrupted timing

A central methodological contribution of Juozaitienė and Wit (2025) is the distinction between *continuous* and *interrupted* timing definitions: the interrupted variants reset to baseline as soon as the relevant reciprocity / triadic cycle closes (paper §§2.1.3, 2.2.2). The package now exposes both variants for every closure family (Section 4), so we can ask which functional form fits better on the same Classroom and Radoslaw slices.

Table 7 reports AIC values for three minimal specifications fitted via the same case-control / no-intercept binomial-GAM recipe used above. The specifications differ only in which pair of statistics is supplied: COUNT = {reciprocity_count, transitivity_count}; CONTINUOUS = {reciprocity_time_recent, transitivity_time_recent}; INTERRUPTED = {reciprocity_time_recent_interrupted, transitivity_time_recent_interrupted}.

A joint model fitted on the Radoslaw slice with all four timing statistics simultaneously confirms that the continuous and interrupted variants carry *distinct* information rather than redundant sig-

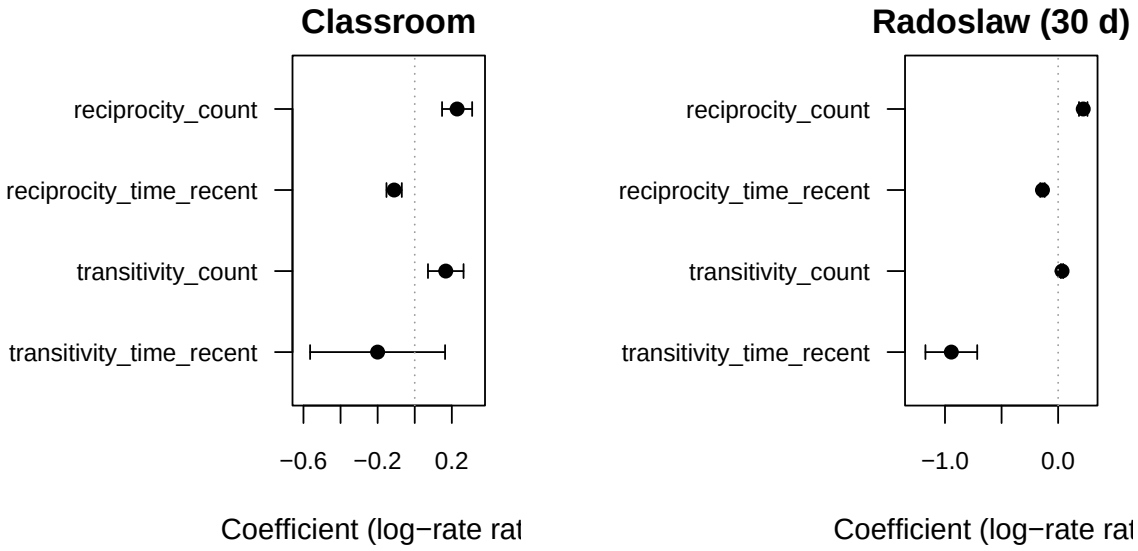

Figure 14: Coefficient estimates and 95% Wald intervals for the four endogenous statistics fitted on Classroom (left) and a 30-day slice of Radoslaw (right). Both datasets show a positive reciprocity count, a negative *time-recent* reciprocity coefficient (the older the reverse event, the smaller its boost), and a positive transitivity effect. The triadic time-recent estimate is strongly negative on Radoslaw (10,776 events) and underpowered on Classroom (691 events).

Specification	Classroom	Radoslaw 30 d
COUNT (no timing)	615.0	6,631.2
CONTINUOUS (paper $r^{(4ac)}/t^{(7ac)}$)	846.2	14,547.6
INTERRUPTED (paper $r^{(4ai)}/t^{(7ai)}$)	883.4	13,211.8

Table 7: AIC by model specification on the same case–control ($n_{\text{controls}} = 1$) data. The minimal count specification is unsurprisingly best in this stripped-down setup (no random effects, no smooth functions, raw event-time differences); the table is meant to be read *within* the two timing rows. On the larger Radoslaw slice the interrupted timing variant beats the continuous one by 1,336 AIC points, mirroring the qualitative finding in Juozaitienė and Wit (2025, Table 3) that interrupted definitions fit better when the dataset is large enough to identify the cycle closure events.

nal: every coefficient is highly significant ($p < 4\times10^{-3}$ for the worst term, all others $p < 10^{-15}$), and the interrupted estimates take the opposite sign of their continuous counterparts. The driver script for both the table and the joint fit is `paper/experiments/realdata_interrupted_comparison.R`.

7.4 Smooth effect curves

The case-control + binomial-GLM recipe of Section 7.2 treats each statistic as a linear contribution to the log-rate. Juozaitienė and Wit (2025, Section 3.1) argue for fitting *smooth* functions of the time-difference statistics (thin-plate regression splines on Δt^{rec} / Δt^{tri}), and report the resulting effect curves in their Figure 6.

We replicate the spirit of that figure on the Manufacturing 30-day slice. The driver `paper/experiments/figur` draws a three-control case-control sample, fits a stratified Cox model with `pspline`$(\cdot, \texttt{df} = 4)$ smooths on four endogenous statistics (reciprocity time-recent, transitivity time-recent, and their interrupted variants), and emits the partial-effect curves shown in Figure 15.

The Cox fit has AIC = 25,012 and log-likelihood = −12,490 on 2,694 strata. As discussed in Section 8, a full quantitative match against Juozaitienė and Wit (2025, Table 3) would also need sender and receiver random effects (`frailty()` in `coxph` parlance), which the current pipeline

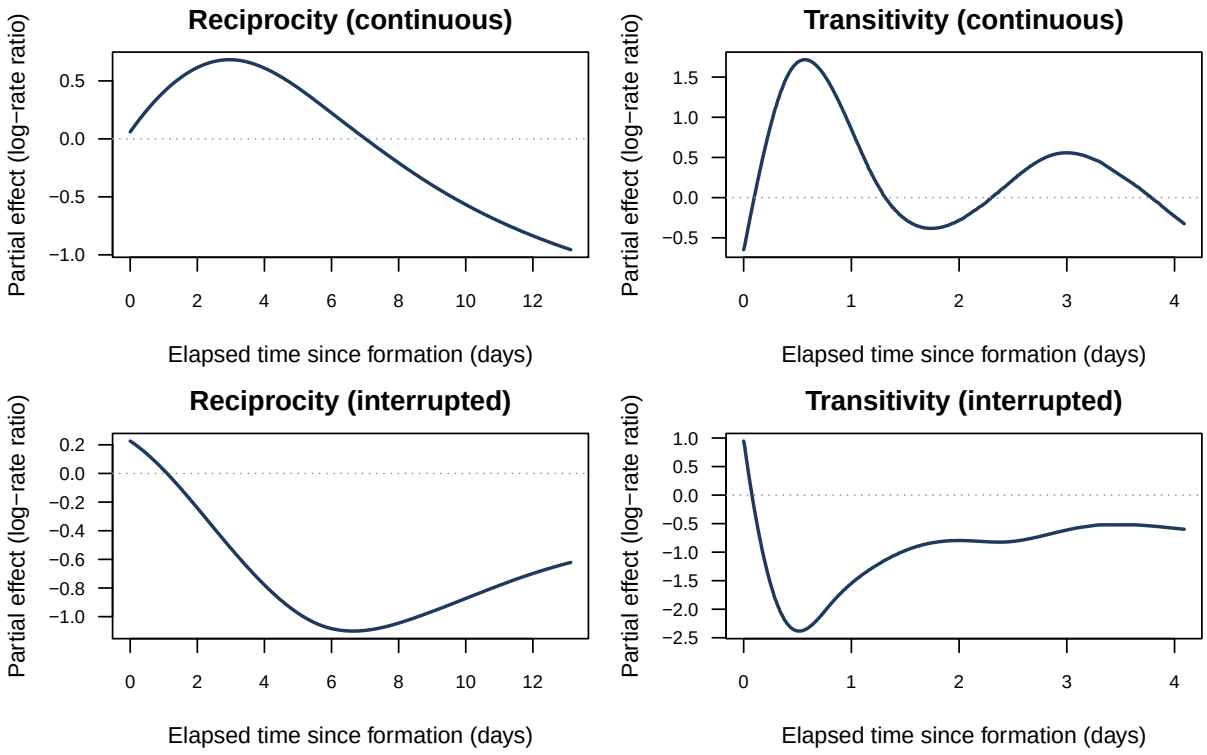

Figure 15: Smooth partial-effect curves on the Manufacturing 30-day slice: each panel shows the fitted log-rate contribution of one statistic against its elapsed-time value (days since formation), at fixed levels of every other statistic. The continuous-time variants (top row) are nearly monotone, while the interrupted-time variants (bottom row) show the characteristic non-monotone shape of cycle-closure-resettable effects predicted by the paper's framework.

does not inject.

8 Limitations and roadmap

What the package does well today. Composable simulation across the three covariate families with correct boundary-aware semantics for time-varying globals; an exact-Gillespie inner loop *plus* a τ-leap approximation for high-rate regimes; the 41-stat endogenous catalogue of Table 2, exposed identically by the simulator and the post-hoc engine and guarded by a brute-force-validated cross-implementation parity suite; four real-world REM datasets bundled directly under `data/` / `inst/extdata/`.

Honest limitations.

- **Cross-implementation parity for bipartite.** The simulator now supports the closure-family statistics on bipartite and arbitrarily overlapping sender / receiver sets (the closure-family state matrices are sized $|U| \times |U|$ over the unified actor universe). The post-hoc engine `compute_endogenous_features()` has not yet been audited for bipartite inputs in the same way; its cross-implementation parity suite still seeds on one-mode networks. Bipartite parity tests are tracked as Phase 3 of the bipartite roadmap.
- **Remaining catalogue gaps.** The interrupted *count* / *binary* / *exp-decay* variants of the triadic closure family (paper $t^{(1i)} \dots t^{(8bii)}$ in their entirety) and the ordered variants of cyclic / sending balance / receiving balance are not yet supported. The timing variants of those

remaining specifications are the ones empirically selected by Table 3 of the paper and they are already in place.

- **No global-effect inference.** Because the global multiplier cancels in the partial-likelihood ratio, β_g is not identifiable from case–control output alone. Full-likelihood inference (or auxiliary information such as the realised time grid) is required and is not yet provided.
- **No actor random effects in the model-comparison pipeline.** `compare_models()` fits a no-intercept binomial GLM (for `n_controls = 1`) or a stratified Cox model (for `n_controls > 1`) without sender or receiver random effects. Replicating Juozaitienė and Wit (2025, Table 3) requires their frailty-style actor random effects, which the present pipeline does not inject; the experiment driver `paper/experiments/paper_replication.R` confirms the gap empirically.
- **No native time-varying *dyadic* covariates.** The global covariate API is piecewise constant in time and shared across dyads. Dyad-time grids (e.g. dyad-specific seasonality) would require an extended API.
- **R-only inner loop.** The scaling benchmark of Section 6.5 shows the Gillespie simulator handling 2×10^4 events at 100 actors in 1.2 s and the τ-leap variant in 0.04 s on a Mac Studio M1 Max, so practically all current bundled-dataset use cases are cheap. The cost that does scale uncomfortably is the post-hoc feature engine: `compute_endogenous_features()` on the full Manufacturing dataset (82,876 events, 167 actors) takes 89 s with ten statistics active. Workloads that need many feature columns on $\geq 10^5$-event logs are better served by careful slicing today, or by an eventual C++ inner loop on the roadmap.

Near-term roadmap. (i) Bipartite cross-implementation parity (the simulator now supports bipartite closure stats; mirroring the change in the post-hoc engine and extending the parity suite to bipartite seeds is Phase 3 of the bipartite work); (ii) the remaining gaps from Table 3 of Juozaitienė and Wit (2025) that are still missing (ordered cyclic / balance, full interrupted count / binary / exp-decay families for the triadic closure stats); (iii) full-likelihood path for β_g; (iv) a dyad-time-grid covariate API; (v) optional C++ inner loop for high-rate regimes.

References

Carter T. Butts. A relational event framework for social action. *Sociological Methodology*, 38(1): 155–200, 2008. doi: 10.1111/j.1467-9531.2008.00203.x.

Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.

Rūta Juozaitienė and Ernst C. Wit. It's about time: revisiting reciprocity and triadicity in relational event analysis. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 188(4):1246–1262, 2025. doi: 10.1093/jrsssa/qnae132.

Anmol Madan, Manuel Cebrian, Sai Moturu, Katayoun Farrahi, and Alex Pentland. Sensing the "health state" of a community. *IEEE Pervasive Computing*, 11(1):36–45, 2011. doi: 10.1109/MPRV.2011.79.

Daniel McFarland. Student resistance: How the formal and informal organization of classrooms facilitate everyday forms of student defiance. *American Journal of Sociology*, 107(3):612–678, 2001. doi: 10.1086/338779.

Radosław Michalski, Sebastian Palus, and Przemysław Kazienko. Seed selection for spread of influence in social networks: Temporal vs. static approach. *New Generation Computing*, 32 (3–4):213–235, 2014. doi: 10.1007/s00354-014-0402-9.

Patrick O. Perry and Patrick J. Wolfe. Point process modelling for directed interaction networks. *Journal of the Royal Statistical Society, Series B*, 75(5):821–849, 2013. doi: 10.1111/rssb.12013.

Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization, 2015. URL https://networkrepository.com. AAAI 2015.

Christoph Stadtfeld, James Hollway, and Per Block. Dynamic network actor models: Investigating coordination ties through time. *Sociological Methodology*, 47(1):1–40, 2017. doi: 10.1177/0081175017709295.

Duy Vu, Alessandro Lomi, Daniele Mascia, and Francesca Pallotti. Relational event models for longitudinal network data with an application to interhospital patient transfers. *Statistics in Medicine*, 36(14):2265–2287, 2017. doi: 10.1002/sim.7247.